Guia rápido para explicar o projeto

Grupo 3 | Checkpoint 1 | Roteiro de fala de aproximadamente 4 a 5 minutos

1 Problema e README

Nosso projeto é o Caçador de Fraudes. Queremos identificar transações suspeitas e comparar o Random Forest com uma Árvore de Decisão. O README é o guia do projeto: apresenta o objetivo, os integrantes, a fonte dos dados, os comandos de execução, os resultados e as limitações. O código e os materiais estão organizados no GitHub para consulta e reprodução.

2 Dataset

Utilizamos o Credit Card Fraud Detection da colaboração ULB/Worldline, disponível no Kaggle. A base original tem 284.807 transações e 492 fraudes, cerca de 0,17%. São 30 entradas e uma coluna de resposta, Class. Time é o tempo desde a primeira transação; Amount é o valor; V1 a V28 são componentes de PCA anonimizados. Class vale zero para normal e um para fraude, e não entra como informação de entrada do modelo.

3 Preparação

Encontramos zero valores ausentes e 1.081 cópias completas adicionais. Removemos essas cópias antes da divisão, ficando com 283.726 registros e 473 fraudes. Separamos 60% para treino, 20% para validação e 20% para teste, preservando a proporção das classes. Os números preliminares são da validação; o teste final continua reservado.

4 Algoritmo e baseline

Uma Árvore de Decisão aprende uma sequência de perguntas sobre as variáveis. O Random Forest reúne várias árvores diferentes: cada uma aprende com uma amostra aleatória com reposição e considera um subconjunto de variáveis em cada divisão. Na implementação do scikit-learn, as probabilidades estimadas pelas árvores são combinadas por média. Usamos 100 árvores. A árvore isolada é o baseline, para verificar se a floresta acrescenta valor.

5 Resultado e referências

Na validação havia 94 fraudes. A floresta identificou 80, deixou passar 14 e gerou 15 falsos alertas. Isso corresponde a recall de 85,11% e precisão de 84,21%. A árvore identificou 75 fraudes, com 285 falsos alertas. São resultados iniciais de uma configuração, não garantia de desempenho futuro. Nossa fundamentação inclui cinco artigos científicos sobre floresta, bagging, avaliação, fraude e implementação em Python.

Analogia para lembrar: uma árvore é um avaliador; uma floresta reúne avaliadores diferentes e combina suas estimativas. A diversidade ajuda a reduzir a instabilidade de depender de uma única árvore. Fontes: Breiman (1996; 2001) e documentação do scikit-learn.

O que mostrar e como responder

Abra o README nesta ordem

Repositório: https://github.com/lipeinacio/random_forest

Seção ou arquivo	O que explicar
Início do README	Problema, objetivo e comparação com uma árvore. Git registra o histórico; GitHub hospeda o repositório; README orienta quem vai usar o projeto.
Dataset e fonte	Origem da base, classes raras, anonimização e instruções para baixar os dados.
Instalação e demonstração	requirements.txt fixa versões; src/baixar_dados.py obtém a base; src/pipeline.py executa o fluxo; src/prever.py faz previsões.
Resultados preliminares	Mostrar a tabela e a matriz de confusão, apontando alertas falsos e fraudes não detectadas.
Limitações e referências	Dados de 2013 e apenas dois dias; cinco artigos científicos e fontes oficiais de apoio.

Os números que você precisa saber

Modelo	Fraudes identificadas	Falsos alertas	F1
Árvore	75 de 94	285	0,3304
Random Forest	80 de 94	15	0,8466

Recall = 80/94: das fraudes reais, quantas identificamos? Precisão = 80/95: dos 95 alertas gerados, quantos eram realmente fraude? F1 combina precisão e recall pela média harmônica. Falso positivo é uma transação normal sinalizada; falso negativo é uma fraude que passou.

Respostas curtas para perguntas prováveis

Por que não usar só acurácia?

Porque responder sempre normal já acerta aproximadamente 99,83% da base original e não identifica nenhuma fraude.

O que é aprendizagem supervisionada?

O modelo aprende com exemplos que já têm a resposta Class conhecida. Depois tentamos prever a classe de outras entradas.

Por que remover duplicados antes de dividir?

Para evitar a mesma entrada no treino e na avaliação. A decisão foi conservadora e está documentada, pois faltam identificadores dos eventos.

O que significa class_weight=balanced?

Aumenta o peso dos exemplos da classe rara no treinamento. Não cria fraudes novas nem torna a base equilibrada.

Vocês fizeram PCA ou sabem o que V17 representa?

Não. Esses componentes vieram prontos. Não conhecemos seu significado comercial original.

O modelo já está pronto para um banco?

Não. O estudo é acadêmico, sem validação temporal, calibração ou teste final. A aplicação precisa receber o mesmo formato de 30 entradas.

O que vem depois?

Comparar pesos de classe, profundidade das árvores e remoção de componentes. Os três novos experimentos estão planejados, ainda não executados.

Demonstração: em uma máquina com as dependências instaladas, execute python src/prever.py na raiz do projeto. O arquivo demo/transacoes.csv fornece dez entradas e o gabarito fica separado. O score não é uma certeza.

Cinco artigos científicos para fundamentar

Os cinco itens abaixo são artigos de periódicos científicos. Dataset e documentação são referências de apoio, mas não foram contados como artigos. Priorize compreender a contribuição dos três primeiros para responder ao professor.

Artigo 1

BREIMAN, L. Random Forests. Machine Learning, v. 45, p. 5-32, 2001.

https://doi.org/10.1023/A:1010933404324

Contribuição: Fundamenta a combinação de árvores e a aleatoriedade nas variáveis. No projeto, justifica estudar uma floresta em comparação a uma árvore isolada.

Artigo 2

SAITO, T.; REHMSMEIER, M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, v. 10, n. 3, e0118432, 2015.

https://doi.org/10.1371/journal.pone.0118432

Contribuição: Mostra por que analisar precisão e recall é importante em classes raras. No projeto, ajuda a interpretar as fraudes encontradas e os alertas falsos.

Artigo 3

DAL POZZOLO, A.; CAELEN, O.; LE BORGNE, Y.-A.; WATERSCHOOT, S.; BONTEMPI, G. Learned lessons in credit card fraud detection from a practitioner perspective. Expert Systems with Applications, v. 41, n. 10, p. 4915-4928, 2014.

https://doi.org/10.1016/j.eswa.2014.02.026

Contribuição: Discute desbalanceamento, mudança dos padrões de fraude e avaliação em contexto real. No projeto, sustenta a análise crítica e os limites da base.

Artigo 4

BREIMAN, L. Bagging predictors. Machine Learning, v. 24, p. 123-140, 1996.

https://doi.org/10.1007/BF00058655

Contribuição: Apresenta a combinação de modelos treinados com amostras bootstrap. Ajuda a explicar a amostragem com reposição e a agregação que fazem parte da floresta.

Artigo 5

PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, v. 12, n. 85, p. 2825-2830, 2011.

https://www.jmlr.org/papers/v12/pedregosa11a.html

Contribuição: Apresenta a biblioteca utilizada para implementar os classificadores e as métricas. A configuração atual do código deve ser conferida na documentação da versão executada.

Fontes oficiais de apoio

Dataset: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Algoritmo: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

README: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes